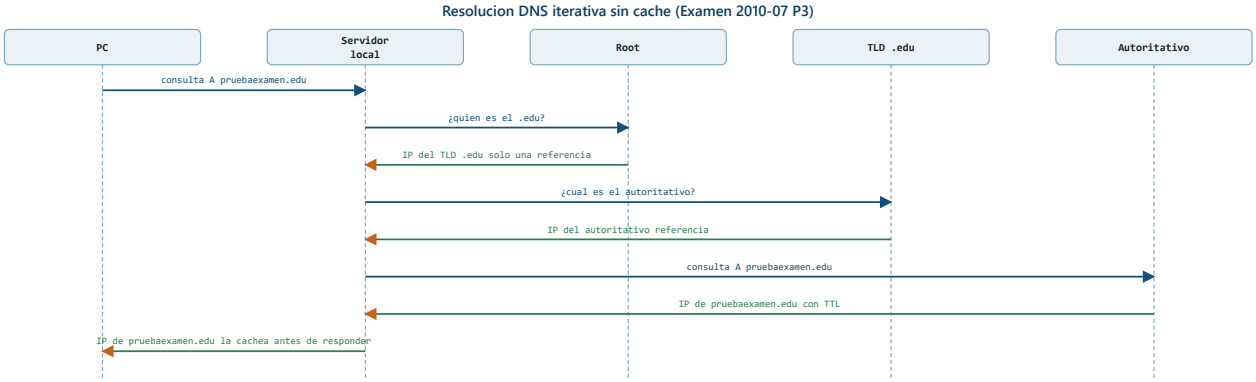
Clase 12 — Taller DNS + HTTP (resuelto)

Bloque: B (Aplicación) · **Semana:** 4 · **Duración estimada:** 110 min **Correlativa:** Clase 11 (DNS: iterativo vs. recursivo, caché y TTL) · **Sigue con:** Clase 13 (correo electrónico: SMTP, POP3, IMAP y MIME) **Material de referencia:** Tanenbaum cap. 7 (§7.4) · RFC 1034, RFC 1035, RFC 6891, RFC 8020, RFC 8767, RFC 9110, RFC 9112 · Exámenes `2010-12` P1, `2010-07` P1 y P3



🎯 Al terminar esta clase vas a poder

1. Calcular **mensajes y RTT** de una resolución DNS en los cuatro escenarios de caché, con la cuenta a la vista.
2. Escribir la **transcripción literal** de los intercambios de una resolución sin caché, distinguiendo **referencia** de **respuesta final** por los flags y los contadores.
3. Escribir un **intercambio HTTP completo que termina en** `304`, y decir qué se ahorra y qué **no** se ahorra.
4. Contar **RTT y conexiones TCP** de una página con 11 objetos, en un servidor o en varios, en persistente y en no persistente, secuencial y en paralelo.
5. Reconstruir **paso a paso** todo lo que pasa al abrir una página: DNS, TCP, TLS, HTTP y las imágenes.
6. Resolver **entero** el Examen `2010-12 P1` y el `2010-07 P3`, detectando dónde la oficial acierta y dónde tiene un error conceptual.

> **Prerequisito del examen:** `2010-12 P1` (8 puntos — ★ el TTL del RR y los pasos de una consulta tipo A), `2010-07 P3` (8 puntos — ★ la resolución iterativa completa sin caché), `2010-07 P1` (8 puntos — HTTP no persistente vs. persistente), `2007-07 P4` (8 puntos — función del DNS y protocolo involucrado), `2010-02 P1b` (iterativa vs. recursiva).
>
> ⚠️ **Aviso de método:** esta clase es **taller puro**. La teoría ya la viste en las Clases 8 a 11. Acá hay **recap de fórmulas**, **once ejercicios resueltos** y **tres problemas de examen**. Si algo del recap te suena raro, volvé a la Clase 11 antes de hacer los ejercicios: el taller no reemplaza la teoría, la usa.

PARTE 1 — RECAP: LA MECÁNICA QUE VAMOS A USAR

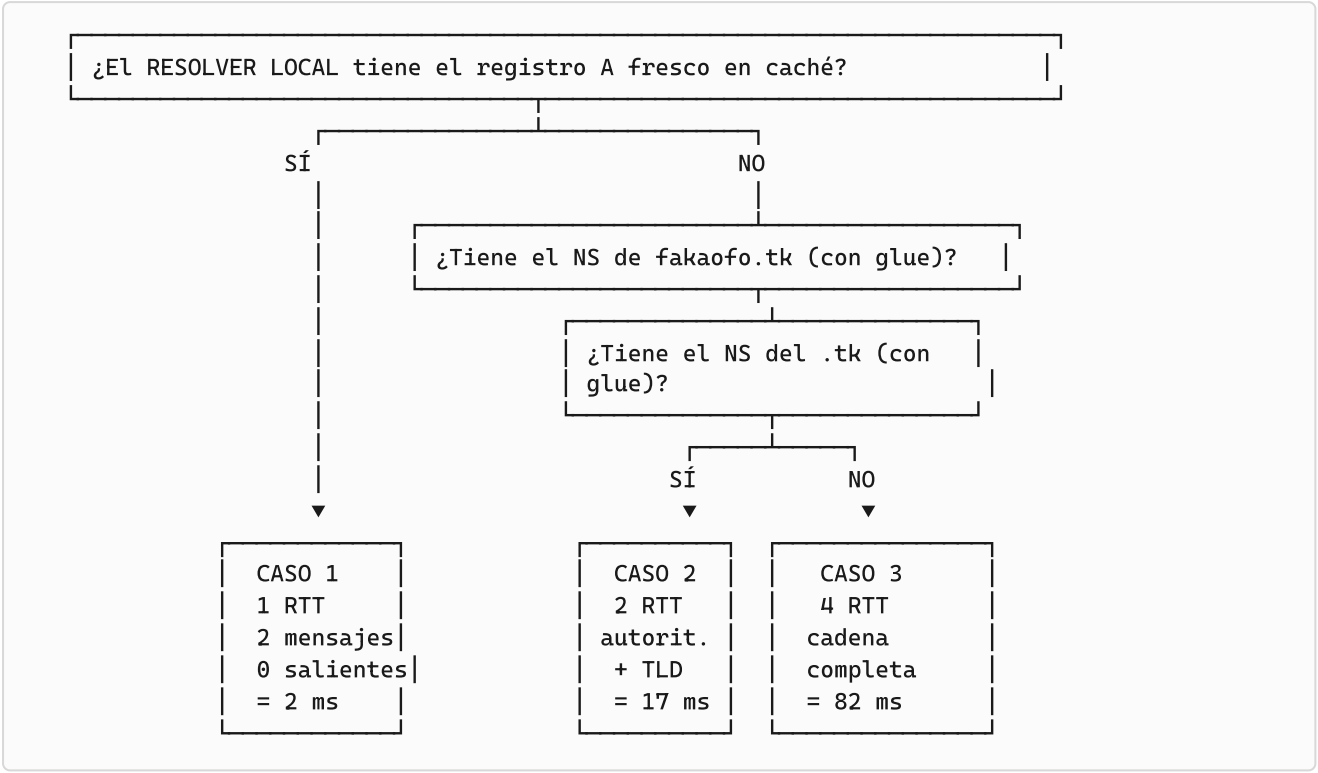
1.1 La caja copiable

Esta caja es lo único que hay que memorizar de esta clase.

```
╔═══════════════════════════════════════════════════════════════════════════╗
║  DNS — LA CAJA DE FÓRMULAS                                                ║
╠═══════════════════════════════════════════════════════════════════════════╣
║  mensajes         = 2 × RTT       (un viaje = ida + vuelta = 1 RTT)       ║
║  RTT              = RTT(PC⇔res) + RTT(res⇔root)                           ║
║                                 + RTT(res⇔TLD) + RTT(res⇔autoritativo)    ║
║                                 ... solo los que se necesiten             ║
║  frescura restante = TTL − Age                                            ║
║  bytes de un A    = 18 + 2 + 2 + 4 + 2 + 4              =  32 bytes       ║
║                     nombre tipo clase TTL rdlength IPv4                   ║
║  acierto de caché  = 0 viajes    ·   falla de caché  = 4 viajes           ║
╠═══════════════════════════════════════════════════════════════════════════╣
║  HTTP — LA CAJA DE FÓRMULAS                                               ║
╠═══════════════════════════════════════════════════════════════════════════╣
║  no persistente     : RTT = N_obj × (1 handshake + 1 petición) = 2·N_obj  ║
║  persistente, 1 srv : RTT = 1 + N_obj                                     ║
║  persistente, k srv : RTT = k + N_obj                 (secuencial)        ║
║  ahorro             = N_obj − k                                           ║
║  conexiones TCP     : N_obj (no persistente) | k (persistente)            ║
║  304                : 0 bytes de cuerpo, pero el RTT se paga igual        ║
╚═══════════════════════════════════════════════════════════════════════════╝
```

Por qué la primera fórmula dice `mensajes = 2 × RTT`. Un **RTT** (*Round-Trip Time*) es una **pareja** de mensajes: uno de ida y uno de vuelta. «El servidor local consulta al root» **son dos paquetes**, y por eso el intercambio completo son **ocho mensajes y cuatro RTT**. El error más común de la unidad va siempre en la misma dirección: **contar de más**.

1.2 El diagrama de decisión: ¿hay caché o no?



> **El atajo, y es lo que hay que decir en voz alta antes de contar: cada nivel que está cacheado en el resolver es un viaje que no se hace.** El resolver no «sabe menos» y contesta igual: **sabe lo mismo**, porque se lo guardó. Lo que cambia es **a quién tiene que preguntarle**. Y la frase que corrige el malentendido más común: **la caché no guarda solamente la respuesta, guarda las DELEGACIONES.** Por eso saltarse el root y el TLD funciona aunque todavía **no** tengas cacheado el `A`.

1.3 Los cuatro flags que hay que mirar

El discriminador entre una referencia y una respuesta no es el lenguaje: son **tres números**. Una **referencia** (la del root y la del TLD) llega con `ANCOUNT = 0` y `AA` ausente, y el dato en **Authority** + **Additional**. La **respuesta final** (la del autoritativo) llega con `ANCOUNT = 1` y `AA = 1`, y el dato en **Answer**.

Bandera	Quién la pone	Qué dice
`QR`	el que responde	0 = consulta, 1 = respuesta
`AA` (*Authoritative Answer*)	el que responde	1 = «esta es la respuesta final, la tengo yo»
`RD` (*Recursion Desired*)	el que pregunta	1 = «si no sabés, averiguatelo vos»
`RA` (*Recursion Available*)	el que responde	1 = «acepto hacer de recursivo»
`TC` (*Truncated*)	el que responde	1 = «no entró, repetí por TCP»

Un **root server** responde siempre con `AA = 0` y `RA = 0`; un **resolver**, con `RA = 1` y `AA = 0` (la respuesta que devuelve al cliente no le pertenece: la tiene cacheada); un **autoritativo**, con `AA = 1`.

1.4 ⚠️ Actualización a 2026

Sigue igual — de memoria: la jerarquía root → TLD → autoritativo; los dos algoritmos; los ocho mensajes y los cuatro RTT; los flags; el TTL como cuarto campo del RR, en segundos; los puertos **53 UDP y 53 TCP**.

Lo que cambió, y acá alcanza con dos cosas. DoH (`RFC 8484`, 2018): si estás en una red con DoH, **el `dig` de la terminal ya no ve lo que ve el navegador** (se verifica en la línea `;; SERVER:`). Y `RFC 6891` **— EDNS(0)**: sin el pseudo-registro `OPT` la respuesta corta en 512 bytes con `TC = 1` y hay que repetir por TCP, y eso **cambia el conteo de RTT**.

> **Consecuencia para el examen:** los ejercicios de esta clase **no cambian en 2026**; lo que se pregunta es la teoría de 1987. Si el enunciado pide el intercambio de mensajes, hacelo como en 2011, y sumá, si te queda lugar, que con DoH ese primer tramo viaja cifrado dentro de HTTPS.

PARTE 2 — TALLER RESUELTO

Mapa de los ejercicios

En la Parte 2 (§2.x): 1, resolución DNS **con** caché comparada con sin caché · 2, intercambio HTTP que termina en `304` · 5, HTTP **no persistente** con 1 HTML + 10 imágenes en uno y en dos servidores · 6, la tabla de **todo** lo que pasa al abrir una página · 7 a 11, los cinco extras (cuatro escenarios de caché, validación condicional con `ETag`, RTT de 20 objetos, TTL `0` y `NXDOMAIN` cacheado). **En la Parte 3 (§3.x):** 3, ★ **Examen** `2010-12 P1` (el TTL y los pasos de una consulta tipo A) · 4, ★ **Examen** `2010-07 P3` (resolución iterativa sin caché de `www.fakaofo.tk`).

2.1 Ejercicio 1 (guía) — resolución DNS con caché en el servidor local

Datos. `necesito.laip.com.uy` consulta a `dns.laip.com.uy` por la IP de `www.fakaofo.tk`. RTT: PC ↔ resolver **2 ms**; resolver ↔ root **40 ms**; resolver ↔ TLD `.tk` **25 ms**; resolver ↔ autoritativo **15 ms**. El resolver **tiene cacheado** el registro `A`, con **TTL = 3600 s**, guardado **hace 600 s**.

a) ¿El registro está fresco?

Razonamiento. La frescura es un contador que baja, y la cuenta va con los números adentro:

```
frescura restante = TTL − tiempo transcurrido
                  = 3600 − 600
                  = 3000 segundos
```

3000 s > 0 ⇒ fresco ⇒ acierto de caché. Si la cuenta diera negativo, habría que revalidar y el ejercicio cambiaría entero.

b) ¿Cuántos mensajes y cuántos RTT?

Son **dos mensajes**: el PC → resolver (la pregunta, recursiva, 2 ms) y el resolver → PC (la respuesta, que sale de la caché). En total, **1 RTT = 2 ms**.

Los mensajes literales (el segundo sale de la caché, sin tocar la red):

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x1A2B
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;www.fakaofo.tk.               IN      A
;; ANSWER:
www.fakaofo.tk.   3000  IN  A   E.F.G.H
;; MSG SIZE  rcvd: 56
```

Mirá el TTL: dice `3000`, no `3600`. Mandó 3600 el autoritativo, pero contesta el resolver, y responde **con la frescura que le queda**.

c) Comparación con el caso sin caché, y cuántos RTT ahorra.

```
SIN NINGUNA CACHÉ (8 mensajes, 4 RTT):
  RTT = 2 + 40 + 25 + 15
      = 82 ms

CON EL A CACHEADO Y FRESCO (2 mensajes, 1 RTT):
  RTT = 2 ms

AHORRO:
  mensajes: 8 − 2  =  6 mensajes menos
  RTT:      4 − 1  =  3 RTT menos
  tiempo:   82 − 2 = 80 ms menos
  porcentaje: 80 / 82 = 0,9756 = 97,6 %
```

Interpretación física: los 6 mensajes ahorrados son los tres viajes hacia la jerarquía; lo único que cruzó el enlace fue la pregunta del PC y su respuesta. Y notá lo que **no** se ahorra: los bytes de la respuesta final **son los mismos**. **La caché ataca el tiempo y la carga, no el tamaño.**

d) ¿Y si pasan 600 segundos más? A `t = 3 600 s` la frescura llega a 0: **vence**, pero **no se borra** (pasa a *stale*). La consulta siguiente tiene que **revalidar**, y como el `NS` de la zona sigue cacheado, eso cuesta `2 + 15 = 17 ms`.

⚠️ Errores típicos

- **Contar 4 RTT cuando el `A` está cacheado** (si la respuesta sale de la caché, no hay ningún viaje a la jerarquía), **contar el TTL del origen en vez del restante**, o decir que «a los 3600 segundos se borra del caché» (no: deja de ser fresco).

2.2 Ejercicio 2 (guía) — intercambio HTTP completo que termina en `304`

Datos. El usuario ya cargó `www.fing.edu.uy` y tiene en su caché `/css/estilo.css`, servido con `Cache-Control: max-age=600`, `ETag: "css-9c1f-6518a1f0"` y `Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT`. Pasaron **40 minutos**: el `max-age` venció, pero el archivo **no cambió**.

Mensaje 1 — la petición, con las dos condiciones:

```
GET /css/estilo.css HTTP/1.1
Host: www.fing.edu.uy
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
Accept: text/css,*/*;q=0.1
Accept-Encoding: gzip, deflate
If-None-Match: "css-9c1f-6518a1f0"
If-Modified-Since: Mon, 05 Oct 2026 09:14:07 GMT
Connection: keep-alive
```

Mensaje 2 — la respuesta:

```
HTTP/1.1 304 Not Modified
Date: Mon, 05 Oct 2026 10:02:41 GMT
ETag: "css-9c1f-6518a1f0"
Cache-Control: max-age=600
Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT
Vary: Accept-Encoding
Connection: keep-alive
```

Y no hay nada más. Ni cuerpo, ni `Content-Length`, ni `Content-Type`.

El razonamiento. El cliente manda la **condición** (`If-None-Match` es la versión del `ETag`; si van las dos, **manda la del `ETag`**, porque distingue dos versiones del mismo segundo y la fecha no). El servidor **compara**, ve que el `ETag` es el mismo y responde `304`: «no te mando el cuerpo, usá la copia que ya tenés». No es un error ni un redireccionamiento, es éxito. El cliente usa su copia local y **no descarga un byte**.

Lo que **NO** lleva un `304` es el cuerpo, y por eso tampoco `Content-Length` ni `Content-Type`. Lo que **SÍ** lleva es el `ETag` y el `Cache-Control`, y este último **renueva la frescura**: el reloj vuelve a cero. Y el **RTT se paga igual**, porque es una respuesta del servidor: el viaje no se evita.

La cuenta de bytes. Cada línea cuenta sin el `CRLF` (2 bytes), más 2 de la línea en blanco que separa cabeceras y cuerpo:

```
DEL 200
"HTTP/1.1 200 OK"                              15
"Date: Mon, 05 Oct 2026 09:14:07 GMT"          35
"Content-Type: text/css; charset=utf-8"        37
"Content-Length: 18422"                        21
"Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT" 50
"ETag: "css-9c1f-6518a1f0""                    25
"Cache-Control: max-age=600"                   26
"Vary: Accept-Encoding"                        21
"Connection: keep-alive"                       22
suma = 252   bytes = 252 × 2 + 2 = 506

DEL 304   (7 líneas: pierde Content-Type y Content-Length)
suma = 25+35+25+26+50+21+22 = 204
bytes = 204 × 2 + 2 = 410

AHORRO:
  antes:  506 + 18 422 de cuerpo = 18 928 bytes
  ahora:  410 +          0        =    410 bytes
  ahorro = 18 928 − 410 = 18 518 bytes   (410 / 18 928 = 2,2 %)
```

Interpretación, y es la frase del ejercicio: el `304` es un **cambio de orden de magnitud** en ancho de banda (pagás el 2 %), pero **el retardo no mejoró nada**. **El `304` ahorra ancho de banda, no retardo**: para matar el retardo hay que eliminar el viaje, y eso lo hacen HTTP/2, HTTP/3 y el CDN.

⚠️ Errores típicos

- **Poner `Content-Length` en el `304`,** decir que **ahorra el RTT**, o creer que **renueva el TTL del origen** (renueva la **frescura en la caché**).

2.3 Ejercicio 5 — HTTP no persistente con 1 HTML y 10 imágenes

Datos. La página tiene **11 objetos** (1 HTML + 10 imágenes). Todos caben en un solo segmento TCP (suposición de la cátedra, y hay que suponerla en voz alta). Cada ida y vuelta cuesta **1 RTT**.

Caso A — los 11 objetos en **un** servidor

```
NO PERSISTENTE
objeto 1 (HTML)    : 1 RTT TCP  + 1 RTT HTTP =  2 RTT
objeto 2 (img 1)   : 1 RTT TCP  + 1 RTT HTTP =  2 RTT
...                                                ...
objeto 11 (img 10) : 1 RTT TCP  + 1 RTT HTTP =  2 RTT
TOTAL = 11 × (1 + 1) = 11 × 2 = 22 RTT
conexiones TCP abiertas = 11

PERSISTENTE
conexión TCP (una sola, para los 11 objetos) :  1 RTT
objeto 1 (HTML)                              :  1 RTT
objetos 2 a 11 (las 10 imágenes)             : 10 RTT
TOTAL = 1 + 11 = 12 RTT
conexiones TCP abiertas = 1

AHORRO = 22 − 12 = 10 RTT = N_obj − k = 11 − 1
```

```
NO PERSISTENTE (22 RTT)      PERSISTENTE (12 RTT)
[TCP][HT] [TCP][HT] [TCP]...  [TCP]-[HT]-[im]-[im]-[im]-----[im]----[im]
 2 RTT   2 RTT   2 RTT          └1┘ └1┘ └──── 10 RTT ──────────┘
  ×11 = 22 RTT                  = 1 + 11 = 12 RTT
```

Interpretación: el no persistente gasta **11 handshakes**, y los handshakes son **tiempo muerto**: no transportan nada útil. El persistente gasta uno y reutiliza la tubería.

Caso B — los objetos repartidos en **dos** servidores

El HTML y 5 imágenes están en **S1** (`www.ejemplo.uy`); las otras 5 en **S2** (`cdn.ejemplo.uy`), otro dominio y por lo tanto **otra dirección IP y otra conexión**. Lo que eso agrega: **una resolución DNS más**. Una página real cuesta de 2 a 6 consultas DNS antes de descargar un solo byte.

Secuencial (un objeto a la vez): `11 × 2 = 22 RTT` no persistente; y con `k = 2` servidores, `2 handshakes + 11 peticiones = 13 RTT` persistente.

En paralelo (lo que hacen los navegadores, que abren varias conexiones):

```
[S1: handshake ── 1 RTT ── GET html ── 1 RTT ──► se ve el HTML]

y EN PARALELO, apenas el navegador parseó el HTML:
[S1: GET img1..img5 ── 1 RTT ──►]   (la conexión de S1 ya estaba abierta)
[S2: handshake ── 1 RTT ──►]  [S2: GET img6..img10 ── 1 RTT ──►]

Los dos renglones del medio corren a la vez, así que el segundo renglón
tarda 2 RTT, no 3.

PERSISTENTE EN PARALELO    = 2 + 2 =  4 RTT
NO PERSISTENTE EN PARALELO = 2 + 2 =  4 RTT   ← ¡empate!
```

Los cuatro escenarios quedan así: secuencial no persistente **22 RTT** con uno o dos servidores; secuencial persistente **12 RTT** con uno y **13 RTT** con dos; en paralelo **4 RTT** en no persistente y **3 RTT** (un servidor) o **4 RTT** (dos) en persistente.

Las dos conclusiones que hay que saber decir, porque son pregunta de examen:

1. **En paralelo empatan o casi.** La ventaja del persistente es **secuencial**: en paralelo los handshakes ya se pagaban a la vez, y lo que queda es el **recurso** (menos sockets y menos puertos efímeros).
2. **Más servidores tienen un costo escondido:** un handshake más (en secuencial) **y una resolución DNS más** (siempre).

2.4 Ejercicio 6 — todo lo que pasa al abrir una página

Datos. RTT = **50 ms** (Montevideo a Estados Unidos). Página **HTTPS** con **1 HTML y 10 imágenes**, todo en **un** servidor, **primera visita** (nada cacheado). Cada objeto pesa **50 KB** y el ancho de banda del cliente es **10 Mbps**.

a) HTTP/1.1 persistente:

#	Etapa	Qué ocurre	RTT	ms
1	**DNS**	4 RTT: resolver → root → TLD → autoritativo, sin caché	**4**	200
2	**TCP**	handshake de 3 vías	**1**	50
3	**TLS**	handshake de TLS 1.3, una ida y vuelta	**1**	50
4	**HTTP**	`GET /index.html` → `200 OK` con el HTML	**1**	50
5	**HTTP**	10 imágenes, persistentes, en la misma conexión	**10**	500
		TOTAL, persistente	**17**	**850**

```
RTT_total = 4 (DNS) + 1 (TCP) + 1 (TLS) + 1 (HTML) + 10 (imágenes) = 17 RTT
tiempo    = 17 × 50 ms = 850 ms
```

b) HTTP no persistente: DNS 4 RTT; el objeto 1 con su handshake 2 RTT; y las 10 imágenes **cada una con su propio handshake**, `10 × (1 + 1) = 20` RTT. Total `4 + 2 + 20 = 26 RTT = 1 300 ms`.

⚠️ **Fijate en el número:** muchos alumnos cuentan 27. El handshake del objeto 1 ya está incluido en los 2 RTT del HTML. La fórmula es `N_obj × (1 + 1)`, y `2 × 11 = 22`, no `2 × 11 + 1`.

c) ¿Cuántas conexiones TCP se abren? En persistente, **1** (que sirve para los 11 objetos). En no persistente, **11**, una por objeto, con sus sockets y sus puertos efímeros. El ahorro son 10 conexiones.

d) Y los bytes, que no son RTT pero también son tiempo. Los datos totales son `11 × 50 KB = 550 KB`, pero **el tiempo que percibís no es el de transmitirlos**: es el que tarda en llegar el **último bit**. El primero tarda `50 KB / 10 Mbps = 40 ms` después de 1 RTT; el último, `40 ms + 10 × 50 ms = 540 ms`.

```
┌──────────────────────────────────────────────────────────────┐
│  850 ms totales                                              │
│    ├─ 200 ms  DNS         (23 %)  ← caché del resolver       │
│    ├─  50 ms  TCP         ( 6 %)  ← reutilización de la conexión │
│    ├─  50 ms  TLS         ( 6 %)  ← reanudación (0-RTT)      │
│    ├─  50 ms  HTML        ( 6 %)                             │
│    └─ 500 ms  imágenes   (59 %)  ← paralelismo y HTTP/2      │
└──────────────────────────────────────────────────────────────┘
```

La moraleja: los RTT dominan la latencia; los bytes no. Es la razón por la que HTTP/2 (`RFC 9113`) y HTTP/3 (`RFC 9114`) no tocan el ancho de banda: **atacan la cantidad de esperas**.

e) ¿Y si fuera la segunda visita? Con el DNS cacheado en el resolver, la conexión viva por `keep-alive`, el TLS reanudado y los recursos con `max-age` fresco, **cada uno de esos puntos vale 0 RTT**: la página entera se ve de memoria.

2.5 Ejercicio 7 (extra) — los cuatro escenarios de caché

Datos. Los del Ejercicio 1: `www.fakaofo.tk` desde `necesito.laip.com.uy` contra `dns.laip.com.uy`, con 2, 40, 25 y 15 ms.

Escenario	Qué tiene cacheado el resolver	A quién consulta	Mensajes	RTT	Tiempo
A	**Nada**	root, TLD, autoritativo	8	4	**82 ms**
B	El `NS` de `.tk` con su *glue*	TLD, autoritativo	6	3	**42 ms**
C	El `NS` de `fakaofo.tk` con su *glue*	autoritativo	4	2	**17 ms**
D	El `A` de `www.fakaofo.tk`, fresco	nadie	2	1	**2 ms**

```
A)  Se saltea nada:          RTT = 2 + 40 + 25 + 15 = 82 ms
B)  Se saltea el root:       RTT = 2 + 25 + 15 = 42 ms
C)  Se saltea root y TLD:    RTT = 2 + 15 = 17 ms
D)  acierto de caché:        RTT = 2 ms
```

Las tres observaciones que valen puntos. (1) **Cada nivel cacheado quita exactamente un viaje:** 4 → 3 → 2 → 1, que es el diagrama de §1.2 con otras palabras. (2) **El salto mayor no es el último** (el `A` ahorra 80 de los 82 ms, la zona 65), pero lo que **mantiene el sistema en pie** es cachear la **delegación**, porque eso se cachea para **millones** de nombres y el `A` hay que cachearlo uno por uno. (3) **El estado normal es el escenario B:** el resolver siempre tiene cacheados los `NS` de los TLD, así que el A casi no existe después del primer arranque.

⚠️ **Ojo con el nombre de los escenarios:** llamarlos «caché en el TLD» es medio mentiroso, porque **la caché está siempre en el resolver**. Lo que se guardó fue la delegación **que el TLD le devolvió**.

2.6 Ejercicio 8 (extra) — validación condicional con `ETag` hasta el `304`

Datos. Un recurso `/datos.json` de **80 000 bytes**, servido con `ETag: "d7a1-4f9c"` y `Cache-Control: max-age=60`. El cliente hace **tres visitas** separadas por **90 segundos**. El recurso cambia entre la segunda y la tercera.

```
Visita 1 (t = 0)    : sin copia local → GET sin condiciones
                      → 200 OK con 80 000 bytes; se guarda cuerpo + ETag

Visita 2 (t = 90 s) : la frescura (60 s) VENCIÓ, pero hay ETag
                      → GET con If-None-Match: "d7a1-4f9c"
                      → NO cambió → 304, 0 bytes de cuerpo → AHORRO: 80 000 bytes

Visita 3 (t = 180 s): la frescura volvió a vencer (el 304 la renovó)
                      → el recurso CAMBIÓ → ETag "d7a1-5110" → 200 OK completo
```

```
petición con If-None-Match = 180 bytes
respuesta 200 = 180 cabeceras + 80 000 cuerpo = 80 180 bytes
respuesta 304 = 120 cabeceras +         0 cuerpo =    120 bytes

Visita 1:  180 + 80 180 = 80 360
Visita 2:  180 +    120 =    300
Visita 3:  180 + 80 180 = 80 360
──────────────────────────────────────────────────────
Total real en 3 visitas        = 161 020 bytes
Sin validación (siempre 200)  : 3 × 80 360 = 241 080
AHORRO = 241 080 − 161 020 = 80 060 bytes  (33,2 %)

Y los RTT: 1 + 1 + 1 = 3, idéntico al de las 3 descargas completas.
```

Interpretación: el `304` no mejora el retardo, solo el ancho de banda. Cambia 80 000 bytes por 120 y deja los RTT donde estaban.

¿Y sin `ETag`? Solo queda la fecha, y `If-Modified-Since` falla si el archivo cambió **dentro del mismo segundo**: el cliente mostraría una versión vieja. El `ETag` identifica el contenido, así que el `304` es siempre correcto, y por eso HTTP le da prioridad a `If-None-Match` cuando están las dos.

2.7 Ejercicio 9 (extra) — RTT de una página con 20 objetos

Datos. 21 objetos (1 HTML + 20 imágenes), 1 RTT por ida y vuelta, 3 servidores.

```
(1) 1 servidor, no persistente, secuencial
    = 21 × (1 RTT handshake + 1 RTT petición) = 21 × 2 = 42 RTT

(2) 1 servidor, persistente, secuencial
    = 1 RTT (un handshake) + 21 RTT (una ida y vuelta por objeto) = 22 RTT
    AHORRO = 42 − 22 = 20 RTT = N_obj − k = 21 − 1

(3) 3 servidores, persistente, en paralelo
    Renglón 1: handshake S1 (1) + GET html (1)                  = 2 RTT
    Renglón 2: los 3 handshakes EN PARALELO (1) + las 20 peticiones
               EN PARALELO (1)                                   = 2 RTT
    TOTAL = 2 + 2 = 4 RTT

(4) 3 servidores, no persistente, en paralelo
    Renglón 1: handshake S1 + GET html                           = 2 RTT
    Renglón 2: handshakes y peticiones, todo en paralelo         = 2 RTT
    TOTAL = 4 RTT   ← IGUAL que el persistente
```

Con 6 conexiones por host, un servidor y paralelismo, el persistente baja a **3 RTT**.

Las tres conclusiones. (1) **El paralelismo importa mucho más que la persistencia:** 42 → 4 RTT paralelizando, 42 → 22 persistiendo. **Orden de magnitud 10 a 1.** (2) **Repartir en más servidores cuesta `k − 1` handshakes extra** en secuencial (`3 + 21 = 24` contra `1 + 21 = 22`) **y una resolución DNS más** por servidor distinto; en paralelo los handshakes extra no cuestan nada. (3) **El límite del paralelismo es el número de conexiones por host:** con 21 objetos y 6 conexiones hacen falta `⌈21 / 6⌉ = 4` tandas, y **cada tanda necesita su propio handshake**: 4 × 2 = 8 RTT, no 4.

2.8 Ejercicio 10 (extra) — ¿qué pasa si el TTL es `0`?

Datos. El autoritativo responde `www.fing.edu.uy. 0 IN A 132.42.120.5`.

```
a) ¿Se puede cachear?
   frescura restante = TTL − Age = 0 − 0 = 0
   0 NO es > 0  →  NO es fresco  →  NO se puede usar de la caché.

b) ¿Qué hace el resolver, entonces?
   Cada consulta tiene que salir a la red:

   PC ⟶ resolver ⟶ autoritativo ⟶ resolver ⟶ PC
   1        2             3             4          5

   4 mensajes, 2 RTT (el NS de la zona sigue cacheado).
   Y no hay validación condicional: en DNS no existe el 304. Cada vez que
   vence, se vuelve a bajar por la jerarquía.

c) ¿Y si el TTL = 0 es el de un NS?
   Peor: el resolver NO PUEDE saber a quién preguntar por el dominio, así que
   va al root Y al TLD en cada consulta: 8 mensajes, 4 RTT. El peor caso
   posible, y casi siempre un error de configuración.
```

¿Tiene sentido un `TTL = 0`? Significa «no me confíes, preguntá cada vez», y tiene sentido si el dato cambia minuto a minuto y uno viejo es **peor** que un viaje extra. Es **error** en un `NS` o «por descuido».

La asimetría que hay que recordar: el `minimum` del `SOA` es el **TTL por defecto de las respuestas negativas**. Ponerlo en `0` **desactiva la caché negativa**, que es la protección contra los ataques por repetición de nombres inexistentes (Ejercicio 11). **El `TTL = 0` en un positivo desactiva el acierto de caché; el `minimum = 0` desactiva la protección.**

2.9 Ejercicio 11 (extra) — por qué el `NXDOMAIN` se cachea

Datos. Un atacante con **un** portátil hace **5 000 consultas por segundo** de nombres inexistentes bajo `fing.edu.uy`, cuya zona tiene `SOA` con `minimum = 3600`.

```
a) SIN caché negativa
   Cada consulta fallida genera 3 viajes:
     resolver → root → resolver → .uy → resolver → fing.edu.uy → resolver

   5 000 consultas/s × 3 viajes = 15 000 paquetes/s contra los servidores de
   nombres de fing.edu.uy, en perpetuo: unos 7,5 MB/s sostenidos desde un
   portátil. Nada para el atacante, la muerte para una zona pequeña.

b) CON caché negativa
   El PRIMER "noexiste.fing.edu.uy" genera un NXDOMAIN cacheado 3600 s.
   Los 4 999 siguientes salen de la tabla:

   1 consulta a la red + 4 999 desde la caché = 0,02 % de la carga original.
   EL DOMINIO SIGUE EN PIE. El ataque es inofensivo.

c) ¿Y 5 000 nombres DISTINTOS por segundo?
   La tabla CRECE sin control: la caché negativa protege contra la REPETICIÓN DEL
   MISMO NOMBRE, y contra la inundación hace falta rate limiting.
```

Lo que dice `RFC 8020` (2018): cachear sí, contaminar no. Cachear la respuesta negativa, **tratarla como un error con nombre** y **guardarla bajo la clave del nombre exacto** que falló. Lo que **prohíbe** es el *domain shadowing*: que el `NXDOMAIN` de `victima.example.com` se derrame sobre `otra.example.com`. Si se guardara «en general», bastaría que un nombre fallara **una** vez para que todos quedaran sombreados, incluidos los que existen: una denegación de servicio silenciosa y hecha con **cero paquetes**.

El resumen del punto: con caché, un nombre que existe y uno que no existe **cuestan 0 viajes** después del primero; sin caché, 4 y 3 viajes por consulta. Al atacante que **repite** lo neutraliza al 99,98 %; al que **inventa** nombres lo deja llenar la tabla, y para eso hace falta *rate limiting*.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-12, Pregunta 1 (8 puntos) — el TTL y los pasos de una consulta tipo A

> **Fuente:** Examen `eirc1012` / `sirc1012` — 17 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1012.pdf

Enunciado (texto literal del examen)

> **Pregunta 1 (8 puntos)**
>
> El DNS es una base de datos distribuida y jerárquica, compuesta por varios tipos de servidores.
>
> **a)** Cuando un servidor de nombres aprende un mapeo, lo cachea. ¿Cuál es el campo del registro de recurso (RR) que decide si se puede cachear o no? ¿Cuál es el servidor que define ese tiempo? ¿El valor que toma ese campo, a qué refiere y en que unidades está expresado?
>
> **b)** Suponga que desde un PC de una empresa, se hace una consulta tipo A al servidor de nombres local por el nombre pruebaexmen.edu, ¿cuales son los pasos siguientes que debe realizar el servidor antes de dar la respuesta al usuario?

(El enunciado escribe `pruebaexmen.edu` en el apartado a) y `pruebaexamen.edu` en el apartado b), y la solución oficial mezcla las dos grafías: es una errata del examen.)

Solución oficial (del PDF, textual)

> **a)** El campo es el TTL, que refiere al tiempo restante por el cual ese RR es válido y por lo tanto se puede mantener en caché. El TTL está definido por el servidor autoritativo de ese RR, y es la cantidad de segundos para los que ese registro es válido. El servidor que lo mantiene en cache, deberá ir decrementando ese valor, hasta que cuando llega a cero, deja de ser válido y se quita del caché.
>
> **b)** PC => Consulta SERVIDOR LOCAL por TIPO A, nombre pruebaexmen.edu SERVIDOR LOCAL => Consulta ROOT SERVER (no lo tiene en caché) por el servidor TLD del .edu ROOT SERVER responde con la IP del servidor TLD del edu SERVIDOR LOCAL => Consulta TLD del .edu por el RR tipo A, nombre pruebaexamen.edu Servidor TDL responde con la dirección IP del nombre pruebaexamen.edu al SERVIDOR LOCAL SERVIDOR LOCAL cachea la respuesta y responde al PC con la dirección IP del nombre pruebaexamen.edu

a) El TTL — resolución

La oficial son cuatro frases y las cuatro son correctas. Lo que hay que hacer es *desarmarlas*, porque cada palabra es medio punto.

Lo que pregunta el enunciado	Lo que responde la oficial	Dónde está
¿Qué campo decide si se puede cachear?	el **TTL**	el **cuarto** de los cinco campos, 4 bytes
¿Qué servidor lo define?	el **servidor autoritativo** de ese RR	quien escribe la zona
¿A qué refiere su valor?	al **tiempo restante** por el cual el RR es válido	contador decreciente
¿En qué unidades?	**segundos**	entero sin signo de 32 bits

```
nombre              clase   tipo    TTL      datos
fing.edu.uy.        IN      A       3600     132.42.120.5
────────────        ─────   ────    ────     ──────────
nombre              2 bytes  2 bytes 4 bytes  variable
                               1       2
                                       ↑
                              EL TTL: 4 bytes, en SEGUNDOS
```

El razonamiento detrás de cada respuesta, que es lo que hay que saber explicar:

1. **¿Por qué el TTL y no otro campo?** Porque los otros cuatro **no dependen del tiempo**. **Solo el TTL declara cuánto tiempo vale lo que dice el registro**, y por eso es el único que puede envejecer.
2. **¿Por qué el autoritativo?** Porque el TTL es una **declaración de política del dueño del dato**, no una decisión técnica de quien lo transporta: si lo definiera el resolver, cada uno tendría su cuenta y el sistema se desincronizaría.
3. **¿Por qué «tiempo restante» y no «fecha de vencimiento»?** Porque es un **contador**: no hay reloj en el protocolo, se manda «3600 segundos» y cada quien lo cuenta **hacia abajo**. Eso permite que dos resolvers en puntos distintos coincidan sin sincronizar relojes.
4. **¿Por qué segundos?** Porque es la **única unidad que el formato del mensaje define**: 4 bytes, entero sin signo de 32 bits. Cualquier otra necesitaría un campo de unidades, que no existe.

> **Nota explícita contra la oficial — y es una diferencia real:** La solución oficial describe **cinco pasos** y en el paso 5 dice que *«Servidor TDL responde con la dirección IP del nombre pruebaexamen.edu al SERVIDOR LOCAL»*. **Mi lectura es que ahí hay un error conceptual:** el servidor del TLD **no puede** responder con la dirección IP, porque **no la tiene y no puede tenerla**: si la tuviera, no habría DNS distribuido, todo el mundo dependería de los TLD y el sistema no escalaría. Lo que el TLD devuelve es el **registro** `NS` que lo delega —quién es el autoritativo— más su *glue*, y **a partir de ahí hay un paso más**: la consulta al autoritativo, que es el único que devuelve el registro `A`. Por eso la cadena completa son **ocho mensajes (cuatro RTT)** y no cinco: es exactamente la misma cadena del `2010-07 P3`. Si el enunciado acepta cinco pasos porque es lo que se da en la clase, **reproducible la tabla oficial** y **agregá una línea con el paso que falta**: eso no te resta nada y muestra que entendiste la jerarquía.

b) Los pasos de una consulta tipo A — resolución

Acá hay que tener cuidado, porque **la solución oficial tiene un error conceptual**. El nombre del problema es `pruebaexamen.edu`: `pruebaexamen` es un dominio de segundo nivel y `.edu` es un **TLD**

genérico (*gTLD*), no el `ccTLD` de un país. La cadena es `raíz → .edu → pruebaexamen.edu → el nombre buscado`.

#	De	A	Qué pasa
1	PC (stub)	servidor local (el de la empresa)	RECURSIVA: "¿IP de pruebaexamen.edu?" RD = 1
2	servidor local	root server	ITERATIVA: la misma consulta
3	root server	servidor local	REFERENCIA: "el TLD .edu está en <IP del .edu>"
4	servidor local	TLD .edu	ITERATIVA: la pregunta
5	TLD .edu	servidor local	REFERENCIA: "el autoritativo es ns1.pruebaexamen.edu, en <IP del autoritativo>"
6	servidor local	autoritativo	ITERATIVA: la pregunta
7	autoritativo	servidor local	RESPUESTA: "es <IP>"
8	servidor local	PC	RESPUESTA: "es <IP>"

```
RTT: 1 (PC↔local) + 1 (local↔root) + 1 (local↔.edu) + 1 (local↔aut.) = 4 RTT
→ 8 mensajes.
```

```
⚠️ EL ERROR DE LA OFICIAL, y hay que verlo:

La oficial dice en su paso 5:
  "Servidor TDL responde con la DIRECCIÓN IP del nombre
   pruebaexamen.edu al SERVIDOR LOCAL"

El TLD .edu NO PUEDE responder con la dirección IP. No la tiene.
Lo único que tiene de pruebaexamen.edu es el registro NS que dice quién lo
autoriza. Si el TLD devolviera la IP, el sistema entero colapsaría en el TLD
y no habría DNS distribuido.

El TLD responde con una REFERENCIA (ANCOUNT = 0), y la respuesta final la da
el AUTORITATIVO en el paso 7.
```

Lo que hace el servidor local antes de responder: los tres `cachear`. Cachea **(1)** el registro `A` con su TTL; **(2)** la referencia del TLD `.edu` —el `NS` y su *glue*— con el TTL del TLD (172 800 s = 2 días), que es lo que hace que la **segunda** resolución del mismo nombre tarde 2 RTT y no 4; **(3)** la referencia del root. **Y recién después responde.**

> **Nota explícita contra la oficial — y es una diferencia real:** La oficial describe **cinco pasos** y en el paso 5 dice que *«Servidor TDL responde con la dirección IP del nombre pruebaexamen.edu al SERVIDOR LOCAL»*. **Mi lectura es que ahí hay un error conceptual:** el TLD **no puede** responder con la dirección IP, porque **no la tiene y no puede tenerla**: si la tuviera, no habría DNS distribuido y todo el mundo dependería de los TLD. Lo que devuelve es el **registro `NS`** que lo delega más su *glue*, y **a partir de ahí hay un paso más**: la consulta al autoritativo, único que devuelve el registro `A`. Por eso la cadena completa son **ocho mensajes (cuatro RTT)** y no cinco: es la misma cadena del `2010-07 P3`. Si el enunciado acepta cinco pasos porque es lo que se da en la clase, **reproducible la tabla**

> **oficial** y **agregá una línea con el paso que falta**: no te resta nada y muestra que entendiste la jerarquía.

3.2 ⭐ Examen 2010-07, Pregunta 3 (8 puntos) — resolución iterativa sin caché

> **Fuente:** Examen `eirc1007` / `sirc1007` — 27 de julio de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1007.pdf

Enunciado (texto literal del examen)

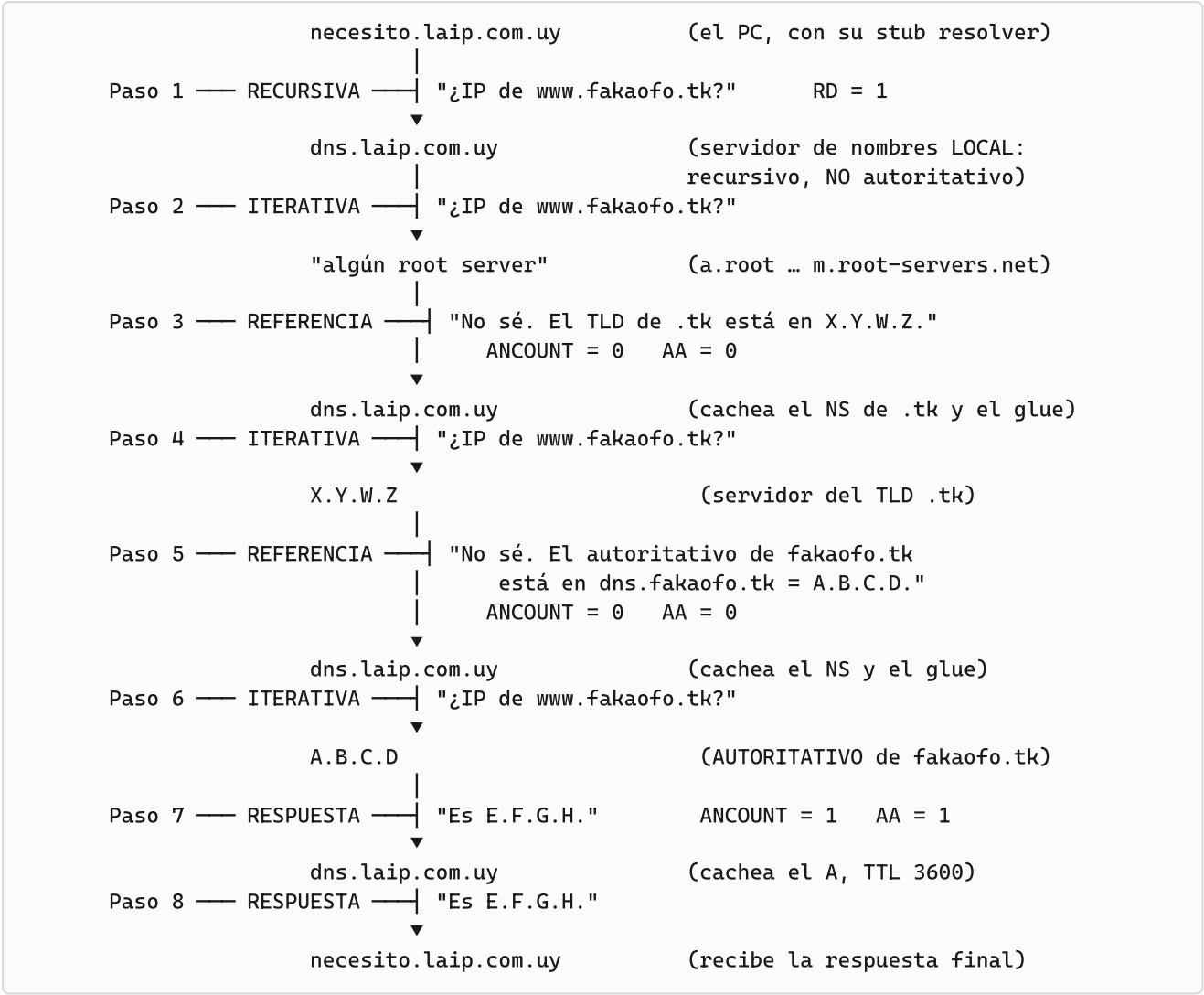
> **Pregunta 3 (8 puntos)**
>
> **a)** Represente en un diagrama todos los pasos involucrados en la resolución de nombres iterativa donde el equipo necesito.laip.com.uy consulta a su servidor DNS local (dns.laip.com.uy) por la dirección IP asociada a www.fakaofo.tk. Suponga que no se dispone de ninguna información de caché en ningún servidor.
>
> **b)** Indique en el diagrama de la parte a) cuál es el servidor del TLD tk y cuál el autoritativo de fakaofo.tk.

Solución oficial (del PDF, textual)

Paso	A	Información
1	necesito.laip.com.uy	dns.laip.com.uy
2	dns.laip.com.uy	"algún root server"
3	"algún root server" (el que recibió la consulta del paso 2)	dns.laip.com.uy
4	dns.laip.com.uy	X.Y.W.Z
5	X.Y.W.Z	dns.laip.com.uy
6	dns.laip.com.uy	A.B.C.D
7	A.B.C.D	dns.laip.com.uy
8	dns.laip.com.uy	necesito.laip.com.uy

> El diagrama debe ser muy similar al que figura en la diapo 9 de la clase 7 de capa de Aplicación. X.Y.W.Z, A.B.C.D y E.F.G.H son meramente ilustrativos, pero sí deben ser distintos.

a) El diagrama, con los cuatro actores



Los cuatro mensajes literales (el `ID` es el mismo en todos porque es **una sola consulta** que viaja por la cadena):

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x1A2B
;; flags: rd; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0
;www.fakaofo.tk.                IN      A

;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x1A2B
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;www.fakaofo.tk.                IN      A
tk.             172800  IN  NS  a.nic.tk.
a.nic.tk.       172800  IN  A   X.Y.W.Z

;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x1A2B
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;www.fakaofo.tk.                IN      A
fakaofo.tk.     172800  IN  NS  dns.fakaofo.tk.
dns.fakaofo.tk.  172800  IN  A   A.B.C.D

;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 0x1A2B
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;www.fakaofo.tk.                IN      A
;; ANSWER:
www.fakaofo.tk.   3600  IN  A   E.F.G.H
```

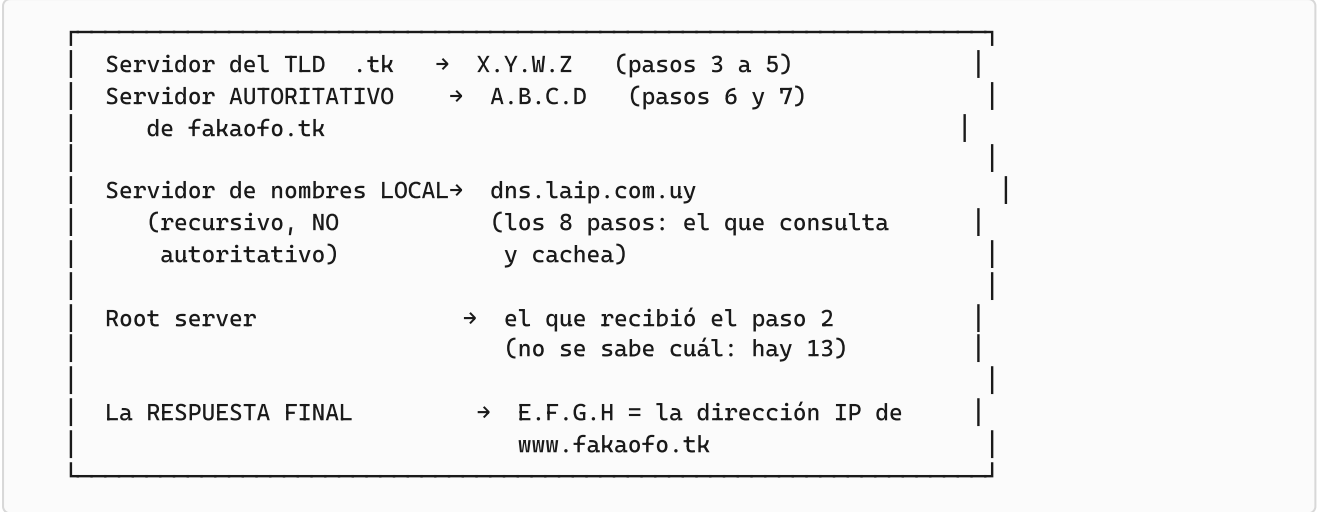
En el laboratorio: `dig +trace www.fakaofo.tk` hace exactamente el recorrido que pide el enunciado; a mano es `dig @a.root-servers.net`, `dig @X.Y.W.Z`, `dig @A.B.C.D` y `dig @dns.laip.com.uy`, siempre con `www.fakaofo.tk A`.

Análisis de la solución oficial. ✅ **Lo que está bien:** (1) **los ocho mensajes están completos**; (2) **los pasos 3 y 5 están escritos como *referencias*, no como respuestas** —«Consultar a TLD de tk», «Consultar a dns.fakaofo.tk» es exactamente la definición de iterativa, y si en alguno de esos dos pasos hubiera puesto «la IP es …», la solución estaría mal; (3) las tres direcciones **son distintas**, como pide el enunciado; (4) **el paso 8 existe**: muchos alumnos lo olvidan y se quedan en 7 pasos.

⚠️ **Lo que la oficial NO dice y hay que agregar:** (i) **qué tipo de consulta es cada paso**: **suma** distinguir que los 3 y 5 son respuestas con `ANCOUNT = 0` y que **solo el 7 tiene** `ANCOUNT = 1` **y** `AA = 1`; (ii) **qué algoritmo es cada paso**: el paso 1 es recursivo en la realidad (ver la nota); (iii) **qué se cachea**: el enunciado dice «sin caché en ningún servidor», pero mencionar qué **se cachearía después** demuestra que entendiste.

> **Nota explícita contra la oficial — y es la más importante de esta clase:** el enunciado dice «**resolución de nombres iterativa**» y describe que `necesito.laip.com.uy` **consulta a su servidor DNS local**. En el modelo real, **esa primera consulta es recursiva**: el *stub resolver* le pide al `dns.laip.com.uy` «resolveste esto por mí y traeme la respuesta final», y el flag `RD = 1` lo dice. Lo **iterativo** son los **cuatro pasos que van del servidor local hacia afuera** (2 a 7). Mi lectura: la oficial usa «iterativa» como abreviatura de «la que hace el servidor local», y **no lo dice**. Si te piden el diagrama, **hacé los ocho pasos** y **agregá la distinción de una línea**: «el paso 1 es recursivo y los 2 a 7 son iterativos». No te resta nada y puede sumarte.

b) ¿Cuál es el TLD y cuál el autoritativo?



(solución propia: el PDF publica la tabla del apartado a) y remite a la diapo 9 de la clase 7 para el dibujo, pero no redacta la respuesta de b).)

El matiz que hay que agregar y es lo que suma puntos: `dns.laip.com.uy` **no es el autoritativo de nada**. Aunque sea el servidor de nombres de `laip.com.uy`, frente a este problema actúa como **resolver recursivo**: es un **cliente** de la jerarquía, no un **dueño** de datos.

3.3 La otra pregunta de HTTP del mismo examen

> **Fuente:** Examen `eirc1007` / `sirc1007`, Pregunta 1 (8 puntos). *(Se resuelve con §2.3.)*

La oficial publica la tabla de pasos del HTTP no persistente —conectar, pedir, recibir, **cerrar** y repetir por cada imagen— y concluye en b): *«En el caso de HTTP persistente, se establece la conexión una sola vez [...]. El uso de HTTP persistente puede resultar más performante en los casos en que se utilice una sola conexión por vez (por ejemplo si el servidor nos limita por IP) [...]. Si el navegador abre conexiones en paralelo para obtener varias imágenes a la vez, probablemente termine de obtener todos los datos más rápidamente, aprovechando mejor el ancho de banda.»*

Resolución: es §2.3 Caso A: **22 RTT y 11 conexiones TCP** en no persistente contra **12 RTT y 1 conexión** en persistente. En b) hay que agregar el matiz que la oficial no cuantifica: **en paralelo empatan** (4 contra 3, y con 3 servidores ambos dan 4), así que la ventaja que le queda es el **recurso** de sockets, no el tiempo. Las dos frases de la oficial que hay que repetir tal cual son la del **paralelismo** y la del **límite de conexiones por IP**.

3.4 Dónde están los puntos de la unidad

Del `2010-12 P1`: **3** para el TTL (cuarto campo, 4 bytes, segundos, definido por el autoritativo, contador decreciente) y **5** para los **ocho** pasos (el TLD devuelve una **referencia**, no la IP; el local **cachea** antes de responder). Del `2010-07 P3`: **6** para el diagrama de **8 mensajes** con los 3 y 5 como *referencias*, y **2** para

identificar `X.Y.W.Z`, `A.B.C.D` y decir que el local no es autoritativo. Del `2010-07 P1`, **4 y 4**: la secuencia de pasos con los 22 RTT, y el empate en paralelo.

PARTE 4 — CIERRE

📌 Resumen

1. `mensajes = 2 × RTT`. Sin caché: 8 mensajes y 4 RTT.
2. Los **3 y 5 son referencias** (`ANCOUNT = 0`, `AA = 0`); solo el **7** es respuesta final (`AA = 1`).
3. **El paso 1 es recursivo** y los 2 a 7 **iterativos**. La oficial no lo dice: decirlo suma.
4. **Cada nivel cacheado quita un viaje**: 4 RTT → 3 → 2 → 1.
5. **La caché ahorra tiempo, no bytes.**
6. **El TTL es el 4.º campo del RR, de 4 bytes, en segundos**; lo define el **autoritativo**, lo **descuenta** quien cachea. `TTL = 0` = «no me confíes».
7. `304` **= 0 bytes de cuerpo y el mismo RTT:** ahorra el 98 % de los bytes y **cero milisegundos**.
8. `RTT = N_obj × 2` no persistente; `1 + N_obj` persistente con un servidor; `k + N_obj` con `k`. El ahorro es `N_obj − k`.
9. **En paralelo, persistente y no persistente empatan.**
10. **Abrir una página HTTPS:** `4 + 1 + 1 + 1 + 10 = 17 RTT` en persistente; `26 RTT` en no persistente, con 11 conexiones. **Los RTT dominan la latencia; los bytes no.**

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Una resolución sin caché tiene 8 mensajes. ¿Cuántos RTT son? ¿Cuáles son *referencias* y cómo lo reconocés?

<details>
<summary>Respuesta</summary>
</details>

2. El resolver tiene cacheado el `NS` de `fakaofo.tk` con su *glue*. Llega una consulta: ¿cuántos mensajes, cuántos RTT y por qué?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Cuáles son los 4 RTT exactos del punto de vista del cliente, y cuánto ahorra la caché con `2`, `40`, `25` y `15` ms?

<details>
<summary>Respuesta</summary>
</details>

4. Un cliente pide un recurso con `If-None-Match` y recibe `304`. ¿Cuánto ancho de banda ahorró? ¿Y cuánto retardo?

<details>
<summary>Respuesta</summary>
</details>

5. ¿Por qué con HTTP persistente el ahorro es exactamente `N_obj − k` RTT, y qué pasa con esa fórmula cuando el navegador abre conexiones en paralelo?

▶ Respuesta

6. Abrís una página HTTPS con 11 objetos a un RTT de 50 ms, en HTTP/1.1 persistente. ¿Cuántos RTT y cuántas conexiones TCP? ¿Qué parte del tiempo se elimina si el DNS está cacheado?

▶ Respuesta

7. ¿Por qué el paralelismo importa más que la persistencia? Pasame el número para 21 objetos y 3 servidores.

▶ Respuesta

8. El autoritativo responde con `TTL = 0`. ¿Qué hace el resolver? ¿Y si además es el `NS` de una zona?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1012` / `sirc1012`** — Pregunta 1 (8 pts): ★ (a) el TTL del RR, quién lo define, a qué refiere y unidades; (b) los pasos de una consulta tipo A
- **Examen `eirc1007` / `sirc1007`** — Pregunta 3 (8 pts): ★ resolución iterativa de `www.fakaofo.tk` sin caché, TLD y autoritativo · Pregunta 1 (8 pts): ★ HTTP no persistente vs. persistente
- **Examen `eirc1002` / `sirc1002`** P1b (iterativas vs. recursivas) · **Examen `eirc0707` / `sirc0707`** P4 (función del DNS y protocolo)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.4
- **RFC 1034** / **RFC 1035** (1987), el diseño del DNS, que **no cambió** · **RFC 6891** — EDNS(0) (2013) · **RFC 8020** — caché negativa por nombre exacto (2018) · **RFC 8767** — servir caché vencida (2020)
- **RFC 8484** — DoH (2018) · **RFC 7858** — DoT (2016) · **RFC 9250** — DoQ (2022) · **RFC 9460** — SVCB/HTTPS (2023)
- **RFC 9110** / **RFC 9112** — HTTP (2022) · **RFC 9113** — HTTP/2 · **RFC 9114** — HTTP/3 · **RFC 9000-9002** — QUIC
- **IANA**, registro de puertos: 53/udp, 53/tcp, 853/tcp (DoT)
- **Clases anteriores:** 8, 9 (HTTP) · 10, 11 (DNS) — **Clase siguiente:** 13 — Correo electrónico: SMTP, POP3, IMAP y MIME